

TECHNICAL REPORT

#1060 on the pipelined FPU configurations: do PR #1065 and PR #1070 hold?

openhwgroup/cv32e40p

Document	RVX-CV32E40P-TRL5-02
Record date	2026-10-03
Prepared by	Rivoryxa Technologies
Classification	Public technical note. Not a verification sign-off.
Website	www.rivoryxatechnologies.com

1 Answer

- Yes, in everything we ran. On F0, F1 and F2 (FPU latency 0, 1, 2), the unfixed RTL shows the stale mstatus.FS read in every tool. With PR #1065, with PR #1070, and with our own comparison patch, the read is correct in every tool.
- The tools:
 - runs under random interrupts, debug requests and bus stalls (up to 8 cycles);
 - random FP programs;
 - 88 directed tests;
 - ACT4.
- FPU latency and memory timing don't matter, and we found no race. The bug and both fixes act at the moment the flw enters write-back, a point neither the FPU latency nor the data-response timing moves.
- The only stale case is the one already reported: an flw followed immediately by csrr mstatus.
- What is not covered: a formal proof, Zfinx under stress, and directed tests with variable memory timing. These are listed under Limits.

2 The question

Matteo Pezzin (CEA), on [#1060](#), [9 September 2026](#) and on [PR #1070](#):

- The two fixes take very different approaches.
- He suspects neither fully fixes the problem, because #1060 looks like a race condition, "in particular ... when the FPU additional pipeline stages are enabled" (the P_F1, P_F2, P_Z1 and P_Z2 configurations).
- CEA is considering dropping those configurations and keeping only F0 and Z0, but wants a status to inform end users.

The two fixes:

- [PR #1065](#) is merged on dev (merge commit dd1f8e4). It keeps the CSR access in decode while an FP load is in execute.
- [PR #1070](#) is open against master (head a68342e). It forwards the Dirty state to the CSR read in the critical cycle.

We applied the rtl/ part of each pull request's diff to master 6033d2b. For comparison only, we also ran our own patch, "local", which stalls in both execute and write-back.

3 What we did

We wrote our predictions down before any run ([PREDICTIONS_2026-10-03.md](#)). Every one held.

tool	what it does	how judged
Directed suite, 88 tests	One FP instruction, then d instructions, then csrr mstatus; d = 0, 1, 2, 8	The test itself checks FS = Dirty and SD = 1
Stimulus runs	A C program with 400 pairs (flw then read, fadd.s then read, flw + addi then read), run under random interrupts, debug requests and bus stalls, 20 seeds	The RTL trace (RVFI) is replayed on Spike, instruction by instruction
Same, harder stalls	Bus stall length up to 8 cycles instead of 3	Same
Random programs	riscv-dv style random programs with mstatus probes after FP instructions: classes fp_mstatus (40 programs), fp_mstatus_grid (64), fp_mstatus_offset (32)	Spike replay
ACT4	Architectural tests, 181 on F, 94 on I0	Self-check

The directed suite covers these instructions:

- Loads and arithmetic: flw, flw from a misaligned address, fadd.s, fmul.s, fmadd.s, fnmsub.s, fdiv.s, fsqrt.s, fmin.s, fsgnj.s.
- Conversions and moves: fcvt.s.w, fcvt.w.s (inexact), fcvt.wu.s (inexact), feq.s (signalling NaN), fmv.w.x, fclass.s, fmv.x.w.
- CSR writes: csrr fcsr, csrr fflags, csrr frm.
- Long operation in flight: fdiv.s or fsqrt.s still running when flw and the read happen.

The 40 tests added on 3 October also read mstatus a second time, 8 instructions later, and fail if the two reads differ.

4 Results

tool	state	F0	F1	F2
Directed suite (88)	unfixed	87 pass, flw_d0 fails	87, flw_d0 fails	87, flw_d0 fails
	#1065 / #1070 / local	88 / 88 / 88	88 / 88 / 88	88 / 88 / 88
Stimulus runs, stalls <= 3, 20 seeds	unfixed	20 DIVERGED	20 DIVERGED	20 DIVERGED
	#1065 / #1070 / local	20 / 20 / 20 MATCH	20 / 20 / 20 MATCH	20 / 20 / 20 MATCH
Stimulus runs, stalls <= 8, 20 seeds	#1065 / #1070	20 / 20 MATCH	20 / 20 MATCH	20 / 20 MATCH
Random fp_mstatus	unfixed	40 / 40 DIVERGED	30 / 30 DIVERGED	30 / 30 DIVERGED
	#1065 / #1070 / local	40 / 40 / 40 MATCH	40 / 40 / 40 MATCH	40 / 40 / 40 MATCH
Random fp_mstatus_grid (64) / fp_mstatus_offset (32)	unfixed	1 / 10 DIVERGED	1 / 10 DIVERGED	1 / 10 DIVERGED
	#1065 / #1070 / local	all MATCH	all MATCH	all MATCH
ACT4	unfixed	180 / 181 (SmF-00 fails)	180 / 181	180 / 181
	#1065 / #1070 / local	181 / 181	181 / 181	181 / 181

- No FPU (I0): ACT4 gives 94/94 with #1065, 94/94 with #1070, and 94/94 unfixed.
- The stale value: the unfixed flw_d0 reads mstatus = 0x00003800 (FS Initial, SD 0). Every fixed state reads 0x80007800.
- Bus stalls reached the case under test. We reran three of the stall-8 seeds with traces kept. They contain 394 back-to-back flw + csrr mstatus pairs, 294 of them in periods with data-response delays switched on, and every one read Dirty. Delays are logged per period, not per transaction, so we cannot show that a delay hit the exact cycle in every pair.

Correction: how many random probes actually ran. Some random probes sit behind a random forward branch and never execute. We counted the executed ones from the traces.

class	probes in the programs	probes executed	executed flw at distance	stale on unfixed	stale on any fix
fp_mstatus	480	339	158	158	0
fp_mstatus_grid	64	45	1	1	0
fp_mstatus_offset	32	22	10	10	0

The counts are the same on F0, F1 and F2. On the fixed states one fewer distance-0 flw probe executes in fp_mstatus (157 instead of 158); we did not investigate that one-probe difference.

- This explains the "10 of 16 offset programs" result we noted earlier. The 6 programs that "did not go stale" never ran their probe; it was not a timing effect.
- No other probe was ever stale, on any state.

Mechanism (from the RTL, consistent with every result above)

- The bug: FS becomes Dirty when the flw enters write-back (regfile_we_wb), before its data comes back. On the unfixed RTL the read is stale only if csr_mstatus completes execute in that same first write-back cycle.
- A late data response hides it. It holds execute, so the read happens later and sees Dirty.
- #1065 keeps the CSR access in decode while the FP load is in execute, so it reaches execute at least one cycle after the update.
- #1070 forwards Dirty to the read in that cycle.
- Neither depends on the FPU latency or on data-response timing. FP operations (as opposed to loads) are already held by the existing csr_apu_stall whenever the FPU latency is non-zero. At latency 0 the result comes back early enough.
- Our local patch's extra write-back stall adds nothing for loads.

Side observation (not #1060, not yet run against Spike)

- The behaviour: the RTL sets FS Dirty for fclass.s and fmv.x.w (fpu_fflags_we_o = apu_valid), even though these instructions change no FP state.
- Why it may matter: judging from Spike's source (insns/fclass.s.h, fmv_x_w.h), Spike does not. If so, a program that reads mstatus after one of them with FS Initial would differ from Spike on every state, fixed or not.
- How we checked so far: our directed tests for these two only check that two successive reads agree. Whether this matters for TRL5 needs confirming against Spike and the spec.

5 What this means for TRL5

- #1060 is not a reason to drop F1, F2, Z1 or Z2. Both fixes clear it on the pipelined FPU configurations, and the mechanism does not depend on FPU latency.
- Zfinx (Z0, Z1, Z2) cannot have this bug. FS is hardwired to zero there (cv32e40p_cs_registers.sv). This holds by construction; we did not run Zfinx under stress with the fixes, beyond 16/16 directed smoke tests on Z0, Z1 and Z2 with each PR.
- Either PR is enough for the configurations tested. The choice between them can rest on review and style, not on pipelined-FPU behaviour.
- One open point is worth a decision: the fclass.s / fmv.x.w behaviour above.

6 How to repeat

Tools: Verilator 5.050, xPack riscv-none-elf-gcc 15.2. The testbench is cv32e40p-dv-review 89ad543 plus [tb_fpu_config_plumbing.patch](#); without it, every build runs at latency 0.

The directed suite, from the root of this repository:

```
cd tests/cv32e40p-fs-hazard && ./build.sh && cd ../cv32e40p-tb-run curl -sL -o pr1065.patch
https://github.com/openhwgroup/cv32e40p/pull/1065.diff curl -sL -o pr1070.patch
https://github.com/openhwgroup/cv32e40p/pull/1070.diff for c in F0 F1 F2; do ./build_tb.sh $c; ./run_elfs.sh
work/$c/obj/Vtb_top ../cv32e40p-fs-hazard/build/.elf ./build_tb.sh $c pr1065.patch; ./run_elfs.sh
work/$c/_pr1065/obj/Vtb_top ../cv32e40p-fs-hazard/build/.elf ./build_tb.sh $c pr1070.patch; ./run_elfs.sh
work/$c/_pr1070/obj/Vtb_top ../cv32e40p-fs-hazard/build/*.elf done
```

build_tb.sh applies only the rtl/ hunks of a patch (#1065 also changes CONTRIBUTING.md). We checked these public scripts against our records:

- flw_d0 on F1 unfixed printed 0x00003800 and failed.
- On F2 with #1065 it printed 0x80007800 and passed.
- PR #1070's current diff is identical to the one we tested.

The stimulus runs and random programs need our replay-against-Spike harness, which is not public. The stimulus program is included as stimulus_program/fs_stale.c so the exposure it creates can be read.

7 Limits

- Simulation evidence only. This is not a formal proof of either PR.
- Zfinx with the fixes has only the 16 directed smoke tests per configuration; no stimulus runs and no random programs.
- The directed suite uses the default testbench memory timing. Variable bus timing was exercised only in the stimulus runs, and only at the level of stall periods (see above).
- The testbench's own directed programs were not run on the PR states.
- The fdiv_flw / fsqrt_flw tests cannot isolate the load path, because the divide itself also sets Dirty.
- FS and SD only. The tests do not check fflags contents, and they always start from FS Initial, never from Clean.
- Pull requests can change. #1070 is open; we tested head a68342e.

8 Files

- tests/cv32e40p-fs-hazard/: gen.py, src/*.S (88 tests), build.sh, link.ld, README.md, PREDICTIONS.md (September), PREDICTIONS_2026-10-03.md, stimulus_program/fs_stale.c
- tests/cv32e40p-tb-run/: build_tb.sh, run_elfs.sh
- patches/tb_fpu_config_plumbing.patch